

Final Project

Introduction

This project will serve as your final assignment for the Functional Reactive Programming class.

Your goal is to design a set of reactive applications that provides an online messaging platform. This application is heavily inspired by the online voice & messaging platform : Discord. You'll have to implement the core functionality of Discord.

This project is based around bonuses.

Completing the main part will grant you a maximum of 10/20.

The last 10 points will be in bonuses that you can choose to increase your potential grade.

It's done by **groups of 4**.

It's **due for the end of February**.

However you'll have **multiple milestones** to ensure that you don't work at the last moment.

Main part : Core messaging platform - 10 pts

You must create services that enables the basic usage of Discord :

- ☐ Creating an account, deleting it, modifying account's information.
- ☐ Authentication & Authorization of users on servers.
- ☐ Creating a server, modifying its information, deleting it
- ☐ Inviting users to servers, allowing users to leave servers
- ☐ Creating channels on servers, modifying channel's information, removing them.
- ☐ Post messages on channels, delete them, see messages of a channel.
- ☐ Tag users on messages, add emoji and basic reactions.
- ☐ Allow admin actions on servers : ban/kick users, delete message written by someone else
- ☐ Send private messages between users

These services must handle a large amount of traffic : thousands of servers, millions of channels, billions of messages.

Your standard basic monolithic application won't work.

Therefore you must **create multiple applications that work together** using patterns seen in class :

- synchronous calls pull-based
- asynchronous calls with push based methods : Websockets, HTTP streaming, gRPC, PubSub

You must at least **have to use the PubSub pattern** (Kafka, Pulsar, RabbitMQ, etc) for one communication channel between two services.

You also must use **at least one of each** reactivity implementations :

- Algebraic effects : Future, IO, ZIO, Kyo
- Actor Model : Pekko, Akka
- Streaming : AkkaStreams, PekkoStreams, FS2, ZIO Streams

Also, you must **provide a basic web interface** to interact with your platform.

You need to provide one **README.md** per application describing them and an **ARCHITECTURE.md** to explain your architecture.

Second part : Bonuses - 19 pts

Roles & Access control - 3 pts

- Give roles to users on a server
- Control channel rights based on these roles

Backups & Archive - 5 pts

- Create an archive of a channel
- Restore an archive to an another channel
- Archive a full server

Improved messaging - 5 pts

- Allow messages to be or to contain media files (images, videos etc)
- Allow messages to tag one or more users using @<username>
- Show "<username> is typing..." when someone's typing in a channel

Statistics & Search - **6 pts**

- Provide statistics by servers & by channels (suggestions : number of posted messages per unit of time, most used keywords, posts leaderboard) - **4 pts**
- Provide search features (by author, in a period of time, containing some words etc..) - **2 pts**

Final notes

- Git is mandatory :
 - Your project must be organized, with a main branch
 - Use Pull/Merge requests to update main branch
 - Your commit messages must be clear and document your changes
 - Do not delete any branches.
- Code should be maintainable, imagine that you're working in a real company. You will be graded like if you were in one.
- HTTP APIs must be RESTful.
- Any LLM or code generation tool is strictly forbidden.
- Each and every member of each group must work on every part.
- You must have your platform deployed in production somewhere to allow me to test it at scale.
Reach me if you want to do it on Clever Cloud.